

Table des matières

Corrigé 02 — Le langage SQL : lire et calculer	1
Questions courtes	1
Questions pratiques	3
Diagnostics	8
Questions à développer	10
Mises en situation	10

Corrigé 02 — Le langage SQL : lire et calculer

Corrigé des exercices de Exercices_02.md. La numérotation est la même.

Questions courtes

1. AS donne un nom à une colonne ou à une expression du SELECT. Ce nom sert d'en-tête au résultat et se réutilise dans le ORDER BY. PostgreSQL le refuse dans le WHERE parce que le WHERE s'exécute avant le SELECT : au moment de filtrer, l'alias n'existe pas encore. DuckDB l'accepte, mais c'est une facilité propre au moteur, qu'on évite dans du code appelé à voyager.

Théorie : Theorie_02.md, section 1.

2. Elle garantit cinq lignes dont le prix est parmi les plus élevés, mais ni lesquelles quand des ex aequo chevauchent la cinquième place, ni leur ordre entre elles : le moteur départage comme il veut, et le résultat peut changer d'une exécution à l'autre. Il faut ajouter une clé de tri qui ne se répète jamais, typiquement la clé primaire : ORDER BY prix DESC, livre_id.

Théorie : Theorie_02.md, section 1.

3. Parce que le texte se compare caractère par caractère, sans égard à la longueur : le premier caractère de « 410 » est « 4 », qui passe devant le « 6 » de « 64 ». Première façon, convertir au moment du tri, ORDER BY CAST(nb_pages AS INTEGER), en protégeant la conversion si certaines valeurs ne sont pas des nombres. Seconde façon, si la valeur est vraiment une quantité, la ranger dans une colonne numérique ; si c'est un identifiant, on ne change pas son type, on le complète de zéros à gauche sur une longueur fixe, et le tri de texte redevient juste.

Théorie : Theorie_02.md, sections 1 et 4.

4. NOT d'abord, puis AND, puis OR. a OR b AND c se lit a OR (b AND c).

Théorie : Theorie_02.md, section 2.

5. Parce que la borne '2026-03-31' est convertie en 2026-03-31 00:00 pour être comparée à un horodatage. Tout ce qui s'est produit le 31 mars après minuit est au-delà de la borne et disparaît. L'intervalle demi-ouvert le remplace : `>= '2026-03-01' AND < '2026-04-01'`.

Théorie : Theorie_02.md, section 2.

6. ILIKE ignore la casse : 'bk-%' trouve BK-M82S-38. Il n'ignore pas les accents : 'aout' ne trouve pas « Août ». Il existe sur PostgreSQL, DuckDB et Snowflake ; ailleurs, on écrit `UPPER(colonne) LIKE UPPER(motif)`.

Théorie : Theorie_02.md, section 2.

7. Le moteur prend le premier WHEN vrai, dans l'ordre d'écriture, et n'examine pas les suivants pour cette ligne. La règle qui en découle : écrire les conditions de la plus particulière à la plus générale, et tester en premier les cas d'exception, comme une sentinelle ou un NULL, qui tomberaient sinon dans une catégorie ordinaire.

Théorie : Theorie_02.md, section 3.

8. La forme simple teste l'égalité `couleur = NULL`, et une comparaison avec NULL n'est jamais vraie, elle est inconnue. On passe à la forme cherchée : `CASE WHEN couleur IS NULL THEN 'aucune' ELSE couleur END`, ou plus court, `COALESCE(couleur, 'aucune')`.

Théorie : Theorie_02.md, section 3.

9. TRY_CAST rend NULL là où CAST ferait échouer la requête ; DuckDB et Snowflake l'ont, PostgreSQL non. Le risque est la perte silencieuse : un NULL produit par une conversion ratée ressemble à une donnée absente, sort des moyennes et des comptes, et personne n'en sait rien. D'où la règle de compter les NULL produits à partir de valeurs qui ne l'étaient pas.

Théorie : Theorie_02.md, section 4.

10. `9 / 12` vaut 0 sur PostgreSQL, qui divise les entiers en entiers, et 0,75 sur DuckDB. `9 * 1.0 / 12`, ou `CAST(9 AS NUMERIC) / 12`, donne 0,75 sur les deux : il suffit qu'un des deux termes soit décimal pour que la division le soit.

Théorie : Theorie_02.md, section 4.

11. `'Jean' || NULL` vaut NULL : concaténé à une absence, tout devient absent. L'écriture portable : `prenom || COALESCE(' ' || second_prenom, '') || ' ' || nom`. Quand le second prénom manque, `' ' || second_prenom` vaut NULL, et COALESCE le remplace par une chaîne vide.

Théorie : Theorie_02.md, section 5.

12. Parce que DATEDIFF(year, ...) compte les passages au 1er janvier entre les deux dates, et non les anniversaires. Toute personne dont l'anniversaire de l'année n'est pas encore arrivé à la date de calcul reçoit un an de trop. Au 1er janvier, cela touche presque tout le monde ; au 31 décembre, presque personne.

Théorie : Theorie_02.md, section 6.

13. COUNT(*) compte les lignes, COUNT(colonne) les lignes où la colonne n'est pas NULL, et COUNT(DISTINCT colonne) les valeurs différentes, NULL exclu.
Sur la table emprunt, pour la colonne amende : 12 lignes, 9 amendes renseignées, 4 montants différents (0,00, 1,00, 1,75 et 3,25).

Théorie : Theorie_02.md, section 7.

Questions pratiques

P1. [OLAP]

```
SELECT EnglishProductName, ListPrice
FROM dbo.DimProduct
WHERE ListPrice IS NOT NULL
ORDER BY ListPrice DESC, EnglishProductName, ProductKey
LIMIT 10;
```

Le résultat : les cinq tailles de Road-150 Red à 3 578,27 \$, les quatre Mountain-100 Silver à 3 399,99 \$, et Mountain-100 Black, 38 à 3 374,99 \$. Le WHERE écarte les 211 lignes sans prix. DuckDB les rangerait de toute façon en dernier, mais PostgreSQL les mettrait en tête d'un tri décroissant : écrit ainsi, le palmarès est juste sur les deux, et ORDER BY ListPrice DESC NULLS LAST ferait le même travail. Le nom départage les ex aequo, et ProductKey termine le travail : DimProduct garde plusieurs versions de certains produits, et un même nom peut s'y répéter.

Théorie : Theorie_02.md, sections 1 et 2.

P2. [OLTP]

```
SELECT name, listprice
FROM production.product
WHERE listprice > 0
ORDER BY listprice DESC, name, productid
LIMIT 10;
```

Exactement les mêmes dix produits, aux mêmes prix. Le filtre change de forme : l'absence de prix est ici la sentinelle 0, pas un NULL, et WHERE listprice IS NOT NULL ne l'écarterait pas. Sans filtre, le tri décroissant les rangerait d'ailleurs en bas et le palmarès serait juste par chance.

Théorie : Theorie_02.md, sections 1 et 2.

P3. [OLTP]

```
SELECT COUNT(*) AS commandes_week_end
FROM sales.salesorderheader
WHERE orderdate >= '2024-01-01' AND orderdate < '2025-01-01'
AND EXTRACT(ISODOW FROM orderdate) IN (6, 7);
```

4 151 commandes sur les 14 244 de 2024. En convention ISO, samedi vaut 6 et dimanche 7 ; avec `EXTRACT(DOW ...)`, il faudrait écrire `IN (0, 6)`, et c'est précisément le genre de détail qu'on se trompe à relire.

Théorie : Theorie_02.md, sections 2 et 6.

P4. [OLAP]

```
SELECT COUNT(*) AS clients
FROM dbo.DimCustomer
WHERE BirthDate >= '1970-01-01' AND BirthDate < '1971-01-01'
AND FirstName ILIKE 'j%';
```

61 clients, sur les 518 nés en 1970. `EXTRACT(YEAR FROM BirthDate) = 1970` donnerait le même compte, mais l'intervalle demi-ouvert laisse la colonne intacte, ce qui compte pour les performances sur un moteur qui utilise des index (Cours 10). `LIKE 'J%'` suffirait ici, les prénoms étant capitalisés ; `ILIKE` répond exactement à la consigne « quelle que soit la casse ».

Théorie : Theorie_02.md, sections 2 et 6.

P5. [OLAP]

```
SELECT CASE TRIM(ProductLine)
    WHEN 'R' THEN 'route'
    WHEN 'M' THEN 'montagne'
    WHEN 'T' THEN 'tourisme'
    WHEN 'S' THEN 'standard'
    ELSE 'sans gamme'
END AS gamme,
COUNT(*) AS lignes
FROM dbo.DimProduct
GROUP BY ALL
ORDER BY lignes DESC;
```

226 lignes sans gamme, 162 route, 112 montagne, 54 standard, 52 tourisme. Sans le `TRIM`, aucune valeur 'R' n'égale 'R', et les 606 lignes tombent dans « sans gamme ». Les `NULL`, eux, vont dans le `ELSE`, puisque `TRIM(NULL)` vaut `NULL` et qu'aucune égalité ne le reconnaît.

Théorie : Theorie_02.md, sections 3, 4 et 7.

P6. [OLTP]

```
SELECT CASE productline
    WHEN 'R' THEN 'route'
    WHEN 'M' THEN 'montagne'
    WHEN 'T' THEN 'tourisme'
    WHEN 'S' THEN 'standard'
```

```

        ELSE 'sans gamme'
    END AS gamme,
    COUNT(*) AS produits
FROM production.product
GROUP BY 1
ORDER BY produits DESC;

```

226 produits sans gamme, 100 route, 91 montagne, 52 tourisme, 35 standard. Deux choses changent dans l'écriture. Le TRIM n'est pas nécessaire, la colonne étant un character qui ignore ses espaces de fin ; le garder ne coûte rien et rend la requête portable, ce que je recommande. Et PostgreSQL n'a pas GROUP BY ALL : on regroupe par position ou en répétant le CASE.

Théorie : Theorie_02.md, sections 3, 4 et 7.

P7. [OLTP]

```

SELECT EXTRACT(YEAR FROM orderdate) AS annee,
       ROUND(100.0 * COUNT(CASE WHEN onlineorderflag THEN 1 END) / COUNT(*), 1) AS pct_en_ligne
FROM sales.salesorderheader
GROUP BY 1
ORDER BY 1;

```

annee	pct_en_ligne
2022	71,3
2023	71,5
2024	88,8
2025	94,6

Le 100.0 placé en tête fait passer tout le calcul en décimal ; placé en fin, COUNT(...) / COUNT(*) * 100.0, il arriverait trop tard, après une division entière qui aurait déjà rendu 0.

Théorie : Theorie_02.md, sections 4, 6 et 7.

P8. [OLAP]

```

SELECT UPPER(LastName) || ', ' || FirstName AS employe
FROM dbo.DimEmployee
ORDER BY LastName, FirstName
LIMIT 5;

```

« ABBAS, Syed », « ABERCROMBIE, Kim », « ABOLROUS, Hazem », « ACKERMAN, Pilar », « ADAMS, Jay ». On trie sur les colonnes d'origine plutôt que sur l'étiquette fabriquée, pour que l'ordre ne dépende pas de la mise en forme. Dans un vrai annuaire, il faudrait aussi écarter les anciennes versions : la table compte 296 lignes pour 290 employés.

Théorie : Theorie_02.md, sections 1 et 5.

P9. [OLTP] (avancé)

```
SELECT COUNT(*) AS employes,
       COUNT(CASE WHEN SPLIT_PART(e.loginid, '\', 2)
                        = SPLIT_PART(a.emailaddress, '@', 1) THEN 1 END) AS concordants
FROM   humanresources.employee AS e
JOIN   person.emailaddress     AS a ON a.businessentityid = e.businessentityid;
```

Les 290 employés concordent. Une concordance parfaite est une propriété des données de démonstration ; sur une vraie base, on comparerait LOWER(...) des deux côtés, parce qu'un identifiant et une adresse ne suivent pas toujours la même casse, et on listerait les discordants plutôt que de les compter. DuckDB accepte la même écriture du séparateur, '\' ; Snowflake, d'après sa documentation, demanderait '\\ '.

Théorie : Theorie_02.md, sections 5 et 7.

P10. [OLAP]

```
SELECT ROUND(AVG(DATEDIFF('year', BirthDate, DATE '2014-01-01')), 1) AS age_naif,
       ROUND(AVG(DATEDIFF('year', BirthDate, DATE '2014-01-01')
                - IF(BirthDate + TO_YEARS(DATEDIFF('year', BirthDate, DATE '2014-01-01'))::INTEGER
                    > DATE '2014-01-01', 1, 0)), 1) AS age_reel
FROM   dbo.DimCustomer;
```

45,2 ans avec DATEDIFF seul, 44,2 ans corrigé. AGE, que DuckDB a aussi, donne directement le même 44,2 : EXTRACT(YEAR FROM AGE(DATE '2014-01-01', BirthDate)). L'écart est presque exactement d'un an parce que la date de référence est un 1er janvier : à cette date, personne n'a encore eu son anniversaire de l'année, sauf les clients nés un 1er janvier. 18 414 clients sur 18 484 reçoivent un an de trop. La même requête au 31 décembre ne se tromperait presque jamais : l'erreur du DATEDIFF dépend de la date choisie, ce qui la rend d'autant plus difficile à repérer.

Théorie : Theorie_02.md, section 6.

P11. [OLTP]

```
SELECT ROUND(AVG(EXTRACT(YEAR FROM hiredate) - EXTRACT(YEAR FROM birthdate)), 1) AS age_naif,
       ROUND(AVG(EXTRACT(YEAR FROM AGE(hiredate, birthdate))), 1) AS age_reel
FROM   humanresources.employee;
```

31,0 ans par soustraction, 30,4 ans avec AGE. L'écart de 0,6 an correspond aux 184 employés sur 290, soit 63 %, qui ont un an de trop : les dates d'embauche sont réparties dans l'année, donc l'erreur ne touche qu'une partie des employés, et la moyenne ne la trahit qu'à moitié.

Théorie : Theorie_02.md, section 6.

P12. [OLAP]

```
SELECT ISODOW(OrderDate) AS jour,  
       COUNT(DISTINCT SalesOrderNumber) AS commandes,  
       SUM(SalesAmount) AS ventes  
FROM dbo.FactInternetSales  
GROUP BY ALL  
ORDER BY jour;
```

jour	commandes	ventes
1, lundi	3 980	4 235 385,79
2, mardi	4 089	4 342 674,03
3, mercredi	3 940	4 231 641,56
4, jeudi	3 915	4 154 919,59
5, vendredi	3 885	4 153 092,89
6, samedi	3 917	4 127 214,75
7, dimanche	3 933	4 113 748,62

Le COUNT(DISTINCT SalesOrderNumber) est indispensable : COUNT(*) compterait des lignes de commande. La semaine est remarquablement plate, avec le mardi légèrement devant. Passer par DimDate et sa colonne DayNumberOfWeek donnerait les mêmes chiffres, mais le dimanche y vaudrait 1, et l'ordre du tri changerait.

Théorie : Theorie_02.md, sections 6 et 7.

P13. [OLTP] (avancé)

```
SELECT CAST(DATE_TRUNC('month', orderdate) AS DATE) AS mois,  
       COUNT(*) AS commandes,  
       ROUND(SUM(subtotal), 2) AS ventes  
FROM sales.salesorderheader  
WHERE orderdate >= '2024-01-01' AND orderdate < '2025-01-01'  
GROUP BY 1  
ORDER BY ventes DESC  
LIMIT 1;
```

Juin 2024 : 741 commandes, 5 063 798,38 \$. Le mois record en montant n'est pas le mois record en nombre : juillet et octobre comptent 1 755 et 1 971 commandes pour un montant moindre. La conversion en DATE évite d'afficher un horodatage à minuit. Enfin, un LIMIT 1 sur un tri qui pourrait comporter des ex aequo mérite une seconde clé ; ici, deux mois au montant identique au cent près sont improbables, mais c'est un choix à assumer, pas à ignorer.

Théorie : Theorie_02.md, sections 1, 6 et 7.

Diagnostics

E1. Elle voit les 211 lignes de DimProduct qui n'ont pas de prix de liste. En ordre décroissant, PostgreSQL range NULL avant toute valeur, et les dix premières places sont prises par des pièces sans prix. DuckDB les range en dernier dans les deux sens, et c'est pourquoi la requête avait l'air juste là où elle a été écrite. La copie est bien chargée ; c'est la requête qui ne dit pas quoi faire des absences. Le correctif, juste sur les deux :

```
SELECT EnglishProductName, ListPrice
FROM dbo.DimProduct
WHERE ListPrice IS NOT NULL
ORDER BY ListPrice DESC, EnglishProductName, ProductKey
LIMIT 10;
```

Théorie : Theorie_02.md, section 1.

E2. [OLTP] Les deux COUNT rendent des entiers, et PostgreSQL divise deux entiers en entiers : 27 659 divisé par 31 465 donne 0, et le reste est jeté. Il faut qu'un des termes soit décimal avant la division :

```
SELECT ROUND(100.0 * COUNT(CASE WHEN onlineorderflag THEN 1 END) / COUNT(*), 1) AS pct_en_ligne
FROM sales.salesorderheader;
```

Le résultat vaut 87,9 %. La même division d'origine aurait rendu 0,879 sur DuckDB, ce qui explique qu'une requête portée d'un moteur à l'autre puisse changer de résultat sans changer d'une virgule.

Théorie : Theorie_02.md, section 4.

E3. [OLAP] Les codes de gamme ont été chargés avec une espace de fin : la valeur rangée est 'M ', et un VARCHAR compte chaque espace à la comparaison. Le moyen de le détecter était de regarder la longueur des valeurs avant d'écrire le filtre, SELECT ProductLine, LENGTH(ProductLine), ou plus simplement de s'étonner d'un zéro sur une gamme qui existe. Le correctif immédiat :

```
SELECT COUNT(*) AS gamme_montagne
FROM dbo.DimProduct
WHERE TRIM(ProductLine) = 'M';
```

112 lignes. Le correctif durable est de nettoyer la colonne une fois pour toutes, au chargement ou par une mise à jour (Cours 03), plutôt que de compter sur chaque auteur de requête pour penser au TRIM.

Théorie : Theorie_02.md, sections 2 et 4.

E4. Tant que la source n'enregistrait que la date, toutes les commandes étaient à minuit, et la borne '2026-03-31' tombait pile dessus : la requête était juste par accident. L'explication la plus probable est un changement en amont, une nouvelle version de l'application ou une migration, qui a commencé à enregistrer l'heure réelle. Depuis, tout ce qui arrive le dernier jour du mois après minuit sort du filtre. Le correctif :

```
WHERE date_commande >= '2026-03-01' AND date_commande < '2026-04-01'
```

Pour confirmer le diagnostic, il suffit de compter les commandes dont l'heure n'est pas minuit, mois par mois : elles apparaissent à partir de la date du changement.

Théorie : Theorie_02.md, sections 2 et 6.

E5. [OLAP] 7 830 enveloppes, soit tous les clients sans second prénom. Leur MiddleName vaut NULL, et concaténé à NULL, le nom complet devient NULL en entier. Le correctif portable :

```
SELECT FirstName || COALESCE(' ' || MiddleName, '') || ' ' || LastName AS nom_complet
FROM dbo.DimCustomer;
```

Sur DuckDB, CONCAT_WS(' ', FirstName, MiddleName, LastName) la sauverait aussi, puisqu'il saute les NULL ; l'écriture par COALESCE a l'avantage de voyager partout.

Théorie : Theorie_02.md, section 5.

E6. [OLAP] DATEDIFF('year', ...) compte les passages au 1er janvier, pas les anniversaires : un employé né en novembre et embauché en juin de l'année de ses 30 ans a un DATEDIFF de 30 et un âge réel de 29. La requête liste 153 employés ; 150 seulement avaient 30 ans ou plus à l'embauche, et trois ont été embauchés à 29 ans. Le correctif retire un an quand l'anniversaire de l'année n'était pas arrivé :

```
SELECT FirstName, LastName
FROM dbo.DimEmployee
WHERE DATEDIFF('year', BirthDate, HireDate)
      - IF(BirthDate + TO_YEARS(DATEDIFF('year', BirthDate, HireDate)::INTEGER) > HireDate, 1, 0) >= 30;
```

Théorie : Theorie_02.md, section 6.

E7. Le moteur prend le premier WHEN vrai. Un produit à 1 500 \$ remplit déjà prix > 100, reçoit « moyen », et la condition prix > 1000 n'est jamais examinée pour lui. L'ordre doit aller du plus exigeant au moins exigeant :

```
CASE
  WHEN prix > 1000 THEN 'cher'
  WHEN prix > 100  THEN 'moyen'
  ELSE 'bon marché'
END
```

Au passage, un prix NULL tomberait dans « bon marché » : si l'absence de prix est possible, elle mérite son propre WHEN prix IS NULL, placé en tête.

Questions à développer

A. L'échec bruyant a pour lui la vérité : une valeur que le moteur ne sait pas lire est une anomalie, et la requête qui s'arrête force quelqu'un à la regarder avant qu'un chiffre faux ne parte dans un rapport. La conversion tolérante a pour elle la continuité : sur une source sale, une ligne mal saisie ne doit pas bloquer le tableau de bord de toute une équipe. Les deux positions se défendent, et c'est pourquoi TRY_CAST n'est acceptable qu'accompagné de sa mesure. J'exigerais que tout code qui l'emploie compte les valeurs non nulles devenues NULL par la conversion, qu'il expose ce compte (une colonne, une table de rejets, une alerte au-delà d'un seuil), et qu'un commentaire dise ce que l'on fait des lignes rejetées. Sans cette mesure, TRY_CAST n'est pas une tolérance, c'est une façon de ne pas savoir.

Théorie : Theorie_02.md, section 4.

B. Entre le 31 décembre 2025 et le 1er janvier 2026, DATEDIFF(year, ...) rend 1 : une frontière d'année a été franchie. AGE rend un jour : presque aucune durée ne s'est écoulée. La frontière est la bonne réponse quand la question porte sur le calendrier lui-même : combien d'exercices financiers une subvention a couverts, à quelle cohorte annuelle un client appartient, combien de fins de mois séparent deux factures mensuelles. La durée est la bonne réponse quand la question porte sur le temps vécu : l'âge légal pour signer un contrat, l'ancienneté qui ouvre droit à une semaine de vacances de plus, la garantie de deux ans d'un appareil. Le piège est d'employer une fonction de frontière pour répondre à une question de durée, parce que les deux rendent le même nombre la plupart du temps.

Théorie : Theorie_02.md, section 6.

C. Pour choisir un type, je pars des opérations que la colonne subira, pas de l'allure de ses valeurs. On calcule dessus : un nombre, exact s'il s'agit d'argent. On l'identifie, on la compare, on la cherche par son début : du texte, même si elle ne contient que des chiffres, et de longueur variable plutôt que fixe, pour ne pas fabriquer d'espaces de rembourrage. On la place dans le temps : une date ou un horodatage, jamais du texte. Pour interroger une colonne dont je n'ai pas choisi le type, la règle est symétrique : je regarde avant de filtrer. Son type, la longueur de ses valeurs et ses valeurs distinctes, la façon dont elle écrit l'absence (Cours 01), et je préfère une comparaison robuste (TRIM, UPPER) ou un nettoyage en amont à la confiance. Le code de gamme et le numéro national racontent la même chose : le type fait partie de la donnée, et l'ignorer donne des résultats faux qui ont l'air justes.

Théorie : Theorie_02.md, sections 1, 4 et 8.

Mises en situation

M1. Le cent perdu vient du FLOAT. Un montant comme 0,10 \$ ne s'écrit pas exactement en binaire, et les petites erreurs de représentation s'additionnent au fil des lignes ; sur PostgreSQL, s'y ajoute un arrondi de la moitié au pair sur ce type, là où la comptable arrondit la moitié vers le haut. Je rangerais les montants en NUMERIC(12, 2), les taux de taxe en NUMERIC(6, 5), et je ne ferais plus aucun calcul d'argent en FLOAT. Pour l'arrondi, je prends position pour une règle écrite et unique : calculer les taxes sur le sous-total de la facture et arrondir une seule fois, parce que c'est ce que fait la comptable et ce que lira le vérificateur, et que l'arrondi ligne par ligne accumule des écarts d'un cent qui ne se compensent pas. Si un règlement impose l'arrondi par ligne, on suit le règlement, mais on le fait

partout de la même façon. Au propriétaire, je réponds que l'ordinateur compte très bien, en base deux, et qu'on lui avait donné un outil de physicien pour faire un travail de comptable.

Théorie : Theorie_02.md, section 4.

M2. Du texte, sans hésiter. En INTEGER, 004217 devient 4217 : les zéros disparaissent à l'insertion et ne reviennent qu'au prix d'un formatage à chaque affichage. Une recherche par début de numéro, LIKE '004%', n'a pas de sens sur un nombre. Et le jour où la fédération imposera un préfixe de lettres, une colonne entière est à migrer. L'argument de la place est négligeable pour quelques milliers de membres, et celui du tri tombe de lui-même : des numéros de longueur fixe, complétés de zéros à gauche, se trient correctement comme du texte. Si un tri numérique était quand même demandé, par exemple pour des numéros de longueur variable, je l'obtiendrais au moment de la requête, ORDER BY CAST(numero AS INTEGER), protégé par un test de forme, sans changer le type de la colonne.

Théorie : Theorie_02.md, sections 1 et 4.

M3. Le rapport filtre avec BETWEEN '...-01' AND '...-31'. Tant que les livraisons étaient à minuit, la borne de fin les incluait toutes. Depuis juillet, une livraison du 31 à 14 h est postérieure à 31 00:00, et elle sort du mois : les fins de mois semblent baisser, alors que ce sont les données qui ont gagné en précision. Avant de toucher au rapport, je le prouverais par deux requêtes : compter, mois par mois, les livraisons dont l'heure n'est pas minuit, qui apparaissent en juillet ; puis comparer, pour chaque mois depuis juillet, le compte avec BETWEEN et le compte avec un intervalle demi-ouvert, dont l'écart doit égaler les livraisons du dernier jour après minuit. La correction remplace le filtre par >= premier_jour AND < premier_jour_du_mois_suivant. Pour protéger les rapports voisins, j'écrirais la règle dans la convention d'équipe, je chercherais dans le code tous les BETWEEN sur des colonnes horodatées, et j'exposerais une colonne ou une vue qui porte la date de livraison convertie en DATE, pour que les usages à la journée n'aient plus à y penser.

Théorie : Theorie_02.md, sections 2 et 6.

M4. (avancé) La normalisation se fait en trois étapes, avec les fonctions du cours. D'abord ne garder que les chiffres, en retirant les séparateurs connus : REPLACE(REPLACE(REPLACE(REPLACE(tel, '(', ''), ')', ''), ' ', ''), '-', ''). Ensuite retirer l'indicatif de pays : CASE WHEN LENGTH(chiffres) = 11 AND chiffres LIKE '1%' THEN SUBSTRING(chiffres, 2, 10) ELSE chiffres END. Enfin reformater à dix chiffres : SUBSTRING(n, 1, 3) || '-' || SUBSTRING(n, 4, 3) || '-' || SUBSTRING(n, 7, 4). Les deux moteurs ont aussi REGEXP_REPLACE, qui retire tout ce qui n'est pas un chiffre en un seul appel, mais la chaîne de REPLACE a le mérite de montrer exactement ce qu'on retire. Pour vérifier la couverture, je compterais les lignes dont le résultat n'a pas dix caractères, ou contient autre chose que des chiffres et deux traits d'union, et je les listerais. Les numéros qui résistent ne se devinent pas : ils partent dans une liste de révision, exclus du dédoublement, plutôt que d'être fusionnés à tort avec un autre client. Et je garderais le numéro d'origine à côté du numéro normalisé, pour trois raisons : pouvoir relancer la normalisation quand on découvre un format oublié, retrouver d'où vient une fusion contestée, et ne jamais perdre une information qu'on ne sait pas encore lire.

Théorie : Theorie_02.md, sections 3, 5 et 7.